EWB EDUTECH PRIVATE LIMITED

CYBERSECURITY

MINI PROJECT WORKBOOK

PHISHING URL AWARENESS CHECKER

COMPLETED PROJECT EDITION

A step-by-step record of the implemented Python checker, its tests, and terminal evidence.

STUDENT	**Vicky Gupta**
BATCH / COURSE	**Batch-1 / Cybersecurity**
LANGUAGE	**Python 3.13.5 | Python 3.10+ compatible**
PROJECT TYPE	**Python command-line application**
DATE	**28 October 2026**

PROJECT REPOSITORY • CLICK TO OPEN

https://github.com/Vickygupta333/Phishing-URL-awareness-checker/tree/main

BUILD • TEST • DOCUMENT • SUBMIT

CONTENTS

Completed workbook

Follow the project from purpose to evidence and handoff.

Sections 00–05

- 00 Start here and project status
- 01 Project brief and key terms
- 02 Tools, Python and project structure
- 03 How URL inspection works
- 04 Build and run the program
- 05 Complete source code

Sections 06–11

- 06 Explain the code
- 07 Test results and sample URLs
- 08 Terminal and code screenshots
- 09 Improvements included
- 10 README and documentation
- 11 Publish and submit checklist

How to read this workbook

This workbook records what is implemented and evidenced in the project folder. Items requiring an external GitHub URL are left as pending because no repository link was supplied.

00 | START HERE

Project journey and status

Defensive learning only: the checker reads text and never opens URLs.

Project stages

- Understand phishing and URL parts
- Implement six text-based warning checks
- Run five workbook samples and automated tests
- Capture source-code and terminal evidence
- Document limitations and ethical use
- Review submission items before publishing

Current evidence

- Working source: phishing_checker.py
- Automated suite: 8 tests passed in the saved terminal capture
- Workbook samples: five cases recorded as passing
- Terminal evidence: screenshots/terminal_commands_proof.png
- Code evidence: screenshots/application_code_proof.png

Project boundary

No live scanning, URL fetching, DNS lookup, or network requests are performed. Findings are clues, not a verdict about safety.

01 | PROJECT BRIEF

What the project does

A beginner Python command-line tool for visible URL warning signs.

Purpose

Phishing messages can use links that look familiar while leading somewhere unexpected. This program lets a learner inspect URL text for selected patterns and see cautious explanations.

Implemented requirements

- Accept a URL as text from the terminal
- Check six visible warning signs and report each one found
- Handle multiple samples in a single session; q, quit, or exit ends it
- Return readable messages for empty, malformed, or hostless inputs
- Show a REVIEW CAREFULLY summary when warning signs are found

What it does not do

It does not establish whether a website is trustworthy, visit a link, detect malware, or check a reputation service. HTTPS by itself does not prove a site is honest.

01 | WORDS YOU WILL MEET

URL parts and project terms

Plain-language reference for explaining the implementation.

URL pieces

- Scheme: how a URL begins, such as http or https.
- Hostname: the name or address identifying the destination, such as www.example.com.
- Path: the requested page, such as /login.
- IP address: a numeric network address, such as 192.0.2.10.

Security terms

- Phishing: deceptive messages or links intended to trick people into sharing information.
- Subdomain: a hostname label placed before a domain and separated by dots.
- Punycode: encoded form for international domain names; xn-- is a common marker.
- Terminal: a text window for running commands and programs.

Project terms

- Function: named block of code that performs a task.
- Test: a check that compares observed behavior with expected behavior.
- Repository: a project stored and versioned on GitHub.

02 | SETUP

Tools and project structure

The implementation needs only Python built-in modules.

Requirements

- Python 3.10 or later; captured session used Python 3.13.5
- VS Code or another text editor
- No third-party Python packages required

Project files

- phishing_checker.py — application
- tests/test_phishing_checker.py — automated tests
- tests/test_samples.md — five sample result table
- README.md — setup, features, use, limitations
- requirements.txt — package note
- screenshots/ — code and terminal evidence

Run from the project folder

Windows: python phishing_checker.py macOS / Linux: python3 phishing_checker.py

Run the automated tests

python -m unittest discover -s tests -v

03 | UNDERSTAND THE IDEA

How URL inspection works

The program treats the supplied URL as text; it never visits it.

Processing flow

- Read a line from the terminal
- Trim leading and trailing whitespace
- Add http:// if the scheme is absent
- Parse URL parts and validate the port
- Inspect hostname and URL text for six signs
- Print findings and wait for another sample

Six implemented checks

- @ symbol anywhere in the candidate URL
- Original URL text longer than 100 characters
- IPv4 or IPv6 hostname recognized by ipaddress
- Five or more non-empty hostname labels
- xn-- substring in the hostname
- Parsed scheme is HTTP

Honest result

The no-warning message says “This does NOT prove safety.” A URL can still be harmful without any of these patterns.

04 | BUILD AND RUN

Program behavior

Improvements in this project add a repeated prompt, cautious summary, and input handling.

Start the application

Open a terminal in the project folder and run: python phishing_checker.py

Example input

http://192.0.2.10/account

Expected warning lines

- Uses an IP address instead of a familiar domain.
- Uses HTTP. HTTPS alone does not prove trust.
- REVIEW CAREFULLY: 2 warning sign(s) found.

Finish the session

Type q, quit, or exit at the next prompt. The program prints a goodbye message.

Input handling

Invalid URL formats, malformed brackets, invalid ports, empty strings, EOF, and Ctrl+C are handled with readable outcomes.

05 | SOURCE CODE | PART 1 OF 3

phishing_checker.py

Exact source from the working project folder

```
001  """A beginner friendly checker for visible URL warning signs.
002
003  This program only reads URL text. It never visits a website or sends data
004  over the network. Its findings are clues, not a verdict about safety.
005  """
006
007  from ipaddress import ip_address
008  from urllib.parse import urlparse
009
010
011  INFO_MESSAGES = (
012      "No URL entered.",
013      "Could not identify a hostname.",
014      "This text is not a valid URL format.",
015      "No listed warning signs found.",
016  )
017
018
019  def inspect_url(raw_url: str) -> list[str]:
020      """Return messages for the warning signs found in URL text."""
021      url = raw_url.strip()
022      findings = []
023
024      if not url:
025          return ["No URL entered."]
026
027      if any(character.isspace() for character in url):
028          return ["This text is not a valid URL format."]
029
030      candidate = url if "://" in url else "http://" + url
031      try:
032          parsed = urlparse(candidate)
033          host = parsed.hostname
```

05 | SOURCE CODE | PART 2 OF 3

phishing_checker.py

Exact source from the working project folder

```
034         parsed.port
035     except ValueError:
036         return ["This text is not a valid URL format."]
037
038     host = (host or "").lower()
039     if not host:
040         return ["Could not identify a hostname."]
041
042     if "@" in candidate:
043         findings.append("Contains an @ symbol; inspect carefully.")
044     if len(url) > 100:
045         findings.append("Unusually long URL (over 100 characters).")
046     try:
047         is_ip_address = bool(ip_address(host))
048     except ValueError:
049         is_ip_address = False
050     if is_ip_address:
051         findings.append("Uses an IP address instead of a familiar domain.")
052
053     labels = [part for part in host.split(".") if part]
054     if len(labels) >= 5:
055         findings.append("Contains many hostname segments/subdomains.")
056     if "xn--" in host:
057         findings.append("Contains a punycode-style hostname.")
058     if parsed.scheme.lower() == "http":
059         findings.append("Uses HTTP. HTTPS alone does not prove trust.")
060
061     if not findings:
062         findings.append("No listed warning signs found. This does NOT prove safety.")
063     return findings
064
065
066 def has_warnings(findings: list[str]) -> bool:
```

05 | SOURCE CODE | PART 3 OF 3

phishing_checker.py

Exact source from the working project folder

```
067     """Tell warning messages apart from input and informational messages."""
068     return bool(findings) and not findings[0].startswith(INFO_MESSAGES)
069
070
071 def main():
072     print("EWB Phishing URL Awareness Checker")
073     print("Text checks only - this program does not open URLs.")
074
075     while True:
076         try:
077             raw_url = input("\nEnter a sample URL (or type q to quit): ")
078         except (EOFError, KeyboardInterrupt):
079             print("\nGoodbye! Stay alert online.")
080             break
081         if raw_url.strip().lower() in ("q", "quit", "exit"):
082             print("Goodbye! Stay alert online.")
083             break
084
085         findings = inspect_url(raw_url)
086         print("\nResult:")
087         for item in findings:
088             print("- " + item)
089         if has_warnings(findings):
090             print(f"\nREVIEW CAREFULLY: {len(findings)} warning sign(s) found.")
091         print("Never enter passwords from unexpected links.")
092
093
094 if __name__ == "__main__":
095     main()
```

06 | THE CODE, BLOCK BY BLOCK

How the implementation is organized

Explain each piece using the real function and behavior names.

1. Imports

ip_address checks whether the parsed hostname is an IP address. urlparse splits URL text into scheme, hostname, port, and other parts.

2. inspect_url(raw_url)

Trims whitespace, rejects empty or whitespace-containing input, adds a default scheme, parses the URL, and returns input errors as messages.

3. Warning checks

The function builds a findings list and appends a message for each matching sign: @, length, IP host, hostname labels, punycode marker, or HTTP.

4. has_warnings(findings)

This helper separates warning findings from informational responses such as empty input, invalid text, and no listed signs.

5. main()

Prints the banner, loops for repeated input, prints results, reports a review count when relevant, and exits cleanly on a quit command or interruption.

07 | TEST YOUR PROJECT

Five workbook samples

Observed results summarized from the saved test evidence.

SAMPLE 1 — PASS

https://www.example.com/login

Expected / observed: No listed warning signs; safety caveat

SAMPLE 2 — PASS

http://192.0.2.10/account

Expected / observed: IP address + HTTP

SAMPLE 3 — PASS

http://example.com@sample.invalid/login

Expected / observed: @ symbol + HTTP

SAMPLE 4 — PASS

https://a.b.c.d.e.example.invalid/path

Expected / observed: Many hostname segments

SAMPLE 5 — PASS

https://xn--example.invalid/

Expected / observed: Punycode-style hostname

Five sample strings use reserved example domains and documentation IP space. The first result is informational and is not a guarantee of safety.

07 | TEST YOUR PROJECT

Automated test result

Captured command: python -m unittest discover -s tests -v

Result

Ran 8 tests in 0.001s — OK

Coverage

- Empty input and informational result classification
- No-warning sample retains an explicit safety caveat
- IP address and HTTP warnings
- @ symbol and HTTP warnings
- Many hostname segments and punycode
- Long URL detection
- Malformed input returns messages without raising
- Empty findings are not classified as warnings

Test implementation

The suite is in tests/test_phishing_checker.py and uses Python unittest. The terminal proof later in this workbook shows the command and passing output.

08 | TERMINAL EVIDENCE

Actual terminal run

Saved capture includes compile check, unittest output, and interactive findings.



```
Problems  Output  Debug Console  Terminal

C:\Users\kisha\Downloads\PHISHING URL AWARENESS CHECKER> python --version
Python 3.13.5

C:\Users\kisha\Downloads\PHISHING URL AWARENESS CHECKER> python -m compileall -q phishing_checker.py tests
[compile completed successfully with no errors]

C:\Users\kisha\Downloads\PHISHING URL AWARENESS CHECKER> python -m unittest discover -s tests -v
test_at_symbol_and_http_are_reported (test_phishing_checker.InspectUrlTests.test_at_symbol_and_http_are_reported) ... ok
test_empty_findings_are_not_warnings (test_phishing_checker.InspectUrlTests.test_empty_findings_are_not_warnings) ... ok
test_empty_input (test_phishing_checker.InspectUrlTests.test_empty_input) ... ok
test_hostname_and_punycode_warnings (test_phishing_checker.InspectUrlTests.test_hostname_and_punycode_warnings) ... ok
test_ip_address_and_http_are_reported (test_phishing_checker.InspectUrlTests.test_ip_address_and_http_are_reported) ... ok
test_long_url_is_reported (test_phishing_checker.InspectUrlTests.test_long_url_is_reported) ... ok
test_malformed_input_is_reported_without_raising (test_phishing_checker.InspectUrlTests.test_malformed_input_is_reported_without_raising) ... ok
test_safe_sample_is_cautious_not_a_verdict (test_phishing_checker.InspectUrlTests.test_safe_sample_is_cautious_not_a_verdict) ... ok

----------------------------------------------------------------------
Ran 8 tests in 0.001s

OK

C:\Users\kisha\Downloads\PHISHING URL AWARENESS CHECKER> python phishing_checker.py
EWB Phishing URL Awareness Checker
Text checks only - this program does not open URLs.

Enter a sample URL (or type q to quit):
Result:
- No listed warning signs found. This does NOT prove safety.
Never enter passwords from unexpected links.

Enter a sample URL (or type q to quit):
Result:
- Uses an IP address instead of a familiar domain.
- Uses HTTP. HTTPS alone does not prove trust.

REVIEW CAREFULLY: 2 warning sign(s) found.
Never enter passwords from unexpected links.

Enter a sample URL (or type q to quit):
Result:
- Contains an @ symbol; inspect carefully.
- Uses HTTP. HTTPS alone does not prove trust.

REVIEW CAREFULLY: 2 warning sign(s) found.
Never enter passwords from unexpected links.

Enter a sample URL (or type q to quit):
Result:
- Contains many hostname segments/subdomains.

REVIEW CAREFULLY: 1 warning sign(s) found.
Never enter passwords from unexpected links.

Enter a sample URL (or type q to quit):
Result:
- Contains a punycode-style hostname.

REVIEW CAREFULLY: 1 warning sign(s) found.
Never enter passwords from unexpected links.

Enter a sample URL (or type q to quit): Goodbye! Stay alert online.
```

The terminal capture records Python 3.13.5, successful compilation, eight passing tests, sample findings, and the quit message.

08 | SCREENSHOTS

Application source evidence

Editor capture from the project screenshots folder.

VS Code - phishing_checker.py

```python
"""A beginner friendly checker for visible URL warning signs.

This program only reads URL text. It never visits a website or sends data
over the network. Its findings are clues, not a verdict about safety.
"""

from ipaddress import ip_address
from urllib.parse import urlparse


INFO_MESSAGES = (
    "No URL entered.",
    "Could not identify a hostname.",
    "This text is not a valid URL format.",
    "No listed warning signs found.",
)


def inspect_url(raw_url: str) -> list[str]:
    """Return messages for the warning signs found in URL text."""
    url = raw_url.strip()
    findings = []

    if not url:
        return ["No URL entered."]

    if any(character.isspace() for character in url):
        return ["This text is not a valid URL format."]

    candidate = url if "://" in url else "http://" + url
    try:
        parsed = urlparse(candidate)
        host = parsed.hostname
        parsed.port
    except ValueError:
        return ["This text is not a valid URL format."]

    host = (host or "").lower()
    if not host:
        return ["Could not identify a hostname."]

    if "@" in candidate:
        findings.append("Contains an @ symbol; inspect carefully.")
    if len(url) > 100:
        findings.append("Unusually long URL (over 100 characters).")
    try:
        is_ip_address = bool(ip_address(host))
    except ValueError:
        is_ip_address = False
    if is_ip_address:
        findings.append("Uses an IP address instead of a familiar domain.")

    labels = [part for part in host.split(".") if part]
    if len(labels) >= 5:
        findings.append("Contains many hostname segments/subdomains.")
    if "xn--" in host:
        findings.append("Contains a punycode-style hostname.")
    if parsed.scheme.lower() == "http":
        findings.append("Uses HTTP. HTTPS alone does not prove trust.")

    if not findings:
        findings.append("No listed warning signs found. This does NOT prove safety.")
    return findings


def has_warnings(findings: list[str]) -> bool:
    """Tell warning messages apart from input and informational messages."""
    return bool(findings) and not findings[0].startswith(INFO_MESSAGES)


def main():
    print("EWB Phishing URL Awareness Checker")
    print("Text checks only - this program does not open URLs.")

    while True:
        try:
            raw_url = input("\nEnter a sample URL (or type q to quit): ")
        except (EOFError, KeyboardInterrupt):
            print("\nGoodbye! Stay alert online.")
            break
        if raw_url.strip().lower() in ("q", "quit", "exit"):
            print("Goodbye! Stay alert online.")
            break

        findings = inspect_url(raw_url)
        print("\nResult:")
        for item in findings:
            print("- " + item)
        if has_warnings(findings):
            print(f"\nREVIEW CAREFULLY: {len(findings)} warning sign(s) found.")
        print("Never enter passwords from unexpected links.")


if __name__ == "__main__":
    main()
```

09 | MAKE IT YOUR OWN

Implemented improvements

The working code includes four concrete changes beyond a one-shot checker.

A Repeat and quit

The prompt repeats so several URLs can be checked in one session. q, quit, and exit end the loop.

B Review summary

has_warnings() identifies actual warning findings; main() prints a review count when any are present.

C Safer URL parsing

Whitespace, malformed URLs, invalid ports, and bracket errors produce readable messages instead of an uncaught ValueError.

D Safer exit

EOF and keyboard interruption produce a goodbye message. No new network behavior is introduced.

Verification

The automated tests and interactive terminal capture provide evidence for these behaviors.

10 | WRITE YOUR README

Project documentation

README.md describes setup, features, testing, limits, and ethical use.

README highlights

- Python 3.10+; no additional packages
- Run with python phishing_checker.py
- Run tests with python -m unittest discover -s tests -v
- Lists all six warning checks
- Explains repeated inputs and malformed input handling
- States clearly that findings do not prove safety

Documentation note

The README currently asks the author to add name and course details before publishing. The report identifies the author as Vicky Gupta, Batch-1 / Cybersecurity.

Screenshot references

The README mentions sample_run.png and test_results.png. Existing screenshot files in the folder use descriptive proof names, including terminal_commands_proof.png and application_code_proof.png.

11 | PUBLISH ON GITHUB

Repository and submission

Repository link supplied by the student

PUBLIC REPOSITORY

Phishing URL Awareness Checker

Owner: Vickygupta333 Branch: main

The project link is included below and on the cover and final handoff page.

OPEN THE GITHUB REPOSITORY

https://github.com/Vickygupta333/Phishing-URL-awareness-checker/tree/main

Final repository review

- Repository link supplied: github.com/Vickygupta333/Phishing-URL-awareness-checker/tree/main
- Use the main branch when reviewing the source and project files.
- Confirm the repository opens for the intended reviewer before submission.
- Keep credentials, access tokens, and private information out of the repository.

Submission note

The URL was added exactly as provided. The workbook links directly to the repository on the main branch.

12 | FINAL CHECK

Deliverables and review checklist

Items are marked from files and evidence available in the supplied project folder.

Complete

- Application source present
- Automated test file present; 8 tests pass in captured terminal evidence
- Five sample results documented; all marked PASS
- README and requirements file present
- Code and terminal screenshots present
- Educational limits and ethics stated

Still to complete

- Add a GitHub repository URL after publishing and verify access
- Replace README author placeholder if desired
- Ensure screenshot filenames linked in README match the files included in the final repository

Evaluation snapshot

Working program: evidenced. Detection logic: six checks implemented. Testing: automated and sample evidence present. Documentation: present. Public repository link: not supplied, so publication cannot be confirmed.

13 | EXPLAIN AND USE SAFELY

Questions, ethics, and troubleshooting

Be ready to explain what the checker sees and what it cannot decide.

Questions to explain

- Does the program open URLs? No, it only parses supplied text.
- Does no warning mean safe? No. The checks cover only six clues.
- Why flag @? It can obscure which hostname a reader expects.
- Why flag HTTP? It does not encrypt the connection; HTTPS still does not prove honesty.

Ethical use

- Use reserved sample strings for practice
- Do not enter passwords or collect credentials
- Do not use it to deceive people or probe sites
- Do not claim it definitively labels links safe or malicious

Common fixes

- If python is unavailable on Windows, try py.
- Run from the folder containing phishing_checker.py.
- For syntax errors, compare indentation and the shown line number.
- For malformed input, the current version should return a readable format message.

14 | FINAL DELIVERABLES

Project handoff and progress tracker

Completed workbook edition

Deliverable	Location / status
Application source	phishing_checker.py • included
README and requirements	Project root • included
Automated tests	tests/ • 8 passing in terminal evidence
Workbook sample table	tests/test_samples.md • included
Code screenshots	screenshots/ • included
Terminal proof	screenshots/terminal_commands_proof.png
GitHub repository	Provided link • included below

FINAL REPOSITORY URL • CLICK TO OPEN

https://github.com/Vickygupta333/Phishing-URL-awareness-checker/tree/main

Student: Vicky Gupta

Batch / Course: Batch-1 / Cybersecurity

Evidence Python version: 3.13.5

Workbook date: 28 October 2026

BUILD WITH CARE. TEST HONESTLY. DOCUMENT CLEARLY. SUBMIT YOUR OWN WORK.